Python and Git 101

Python Programming and Data Analysis · Day 1 · Dataminded Academy

dataminded

NICE TO MEET YOU

Jan Vanbuel

Your trainer today. I like Rust, playing the saxophone and triathlon.

Platform engineer and Dataminded Academy lead



Three days: write it, analyse with it, trust it

DAY 1 — today

Python and Git 101

Write a script and share it

- Python on your laptop, uv
- The language, files, errors
- Git: commit, branch, pull request

DAY 2

Data analysis

Turn measurements into answers

- pandas and NumPy
- SciPy and scikit-learn
- Plots with Matplotlib

DAY 3

Code you can trust

Quality, automation and agents

- Tests, linting, type checking
- Pipelines on Azure DevOps
- Coding agents, and reviewing their work

Bring your own work. A use case, an example or a question from your job is welcome in every session.

Learning in the age of AI

AI is a multiplier of experience.

your experience × AI = what you get

Zero experience gives AI nothing to multiply.

Speak the language

Know the words a specialist uses, so you can ask for exactly that

Understand the tradeoffs

There are several ways to do it, each with a cost. You choose

Ask specific questions

A precise request gets code you can read and check, and makes AI workflows faster and cheaper

This prompt will make sense after the Python part

```
Write a function `summarize_run(path: Path) -> dict` that reads a CSV
log with `csv.DictReader`, converts `pressure_bar` to float, skips and
counts rows that raise `ValueError`, and returns the mean and max. Use
a list comprehension where it fits. Standard library only, no classes.
Add a pytest test using `pytest.approx`.
```




Tonight you can do six things on your own laptop

Run a script

Set up Python and run a script from PowerShell

Manage a project

A virtual environment per project, packages with uv

Write a program

Functions, conditions, loops and imports, run from VS Code

Read an error

Find the line a traceback points to

Automate files

Read a folder of files and write a summary

Work with Git

Clone, commit on a branch, push and pull from a remote

Check the tools, then clone the repository

1 · The tools answer

```
uv --version
uv python list --only-installed
code --version
git --version
```

2 · The course material is on your laptop

```
cd $HOME\Documents
git clone https://github.com/datamindedacademy/ac-python-and-git-101
cd ac-python-and-git-101
uv sync
code .
```



Something fails?

Do not debug alone. Call me over: setup problems are normal on managed laptops.



Python on your laptop



Python fills the gap between shell scripts and C

Shell scripts

Quick to write. Painful once you need real logic, numbers or data structures

Python

Reads like a script, with lists, dicts, functions and a large library built in

C

Can do everything. Slow to write, and easy to get wrong



Photo: Daniel Stroud, CC BY-SA 4.0

Guido van Rossum started it in 1989 at CWI in Amsterdam, for system administration work that was too much for a shell script and not worth a C program. First release in 1991.

Python is the language, CPython runs it

Your .py file → **Compiled to bytecode** → **Run by the virtual machine**

The **interpreter** is the program that does the last two steps: `python.exe` .
To view the bytecode of a script: `uv run python -m dis <script.py>`

	CPython	PyPy	MicroPython
What it is	The reference implementation, written in C	Compiles hot loops to machine code while running	A small Python for microcontrollers
When you meet it	Always: it is what uv installs	Long pure-Python calculations	Sensors and embedded boards



Check which Python a command really runs

```
Get-Command python
```

```
CommandType  Name        Source
-----------  ----        ------
Application  python.exe  C:\Users\you\AppData\Local\Microsoft\WindowsApps\python.exe
```

Watch out

A `WindowsApps` path is the Microsoft Store shortcut, not a real Python. It opens the Store or does nothing useful.

PATH is the list of folders Windows searches, top to bottom, when you type a command.



uv is one tool for Python, packages and running

1

INSTALLS PYTHON

`uv python install 3.13` puts a real CPython in your user folder, no admin rights needed

2

INSTALLS PACKAGES

`uv sync` reads the project file and creates `.venv`, a **virtual environment**: a folder with the packages this project needs

3

RUNS YOUR CODE

`uv run python script.py` uses that Python and those packages, every time

> You did steps 1 and 2 in the setup check. Step 3 is how we run everything today. Creating your own project with uv comes this afternoon.

Go to the repository, then uv run the script

```
# exercises/python/01-first-script/hello.py
print("Hello, test engineer!")
```

```
cd $HOME\Documents\ac-python-and-git-101
uv run python exercises/python/01-first-script/hello.py
```

```
Hello, test engineer!
```

> Paths are read from your **working folder**, the folder PowerShell is in. Check it with `pwd`.



VS Code runs the same script in three steps

1

OPEN THE FOLDER

File, Open Folder, choose `ac-python-and-git-101`

2

SELECT THE INTERPRETER

Command Palette, Python: Select Interpreter, pick the one in `.venv` or the uv Python 3.13

3

RUN

Play button at the top right, or the same `uv run python` command in the integrated terminal



Scripts and notebooks



A script is a text file that runs top to bottom

```
# bench_summary.py
pressure_bar = 7.5

power_kw = 37.5
print(f"Power: {power_kw} kW at {pressure_bar} bar")

print("Done")
```

```
Power: 37.5 kW at 7.5 bar
Done
```

A notebook runs code cell by cell

```
In [1]    pressure_bar = 7.5
```

```
In [2]    pressure_bar * 14.5038
```

```
Out [2]   108.7785
```

```
In [3]    print(f"{pressure_bar} bar")
```

```
Out       7.5 bar
```

- A file with the extension `.ipynb`, opened in VS Code or Jupyter
- You run one **cell** at a time with `Shift+Enter`, in any order
- The **kernel** is the Python process behind it. It keeps your variables between cells
- Output, tables and plots are saved in the file, next to the code



Scripts repeat, notebooks explore

	Script (.py)	Notebook (.ipynb)
Runs	Start to finish, every time, on demand or scheduled	Cell by cell, in any order you click
Great for	Automation, reusable tools, anything others run	Exploring data, plotting, explaining results
Weak at	Slow feedback when exploring	Repeatability and testing
In Git	Clean text, easy to compare versions	JSON with embedded output, noisy differences



A notebook has hidden state

```
In [1]   limit_bar = 9.0
```

```
In [2]   ~~limit_bar = 6.0~~
```

ran, then deleted

```
In [3]   print(limit_bar)
```

```
Out      6.0
```

not 9.0

- The **kernel** remembers every cell you ran, also the one you deleted. That memory is the **hidden state**
- Cell 3 prints a value that appears nowhere in the visible code
- A colleague runs the notebook top to bottom and gets a different answer: it is not **reproducible**
- Restart the kernel and run all cells before you trust a result

Exercise 01: First script

`exercises/python/01-first-script`

Words you can now use script · notebook · cell · kernel · hidden state · reproducible · interpreter · CPython · terminal · PATH · working folder · uv run

Language fundamentals

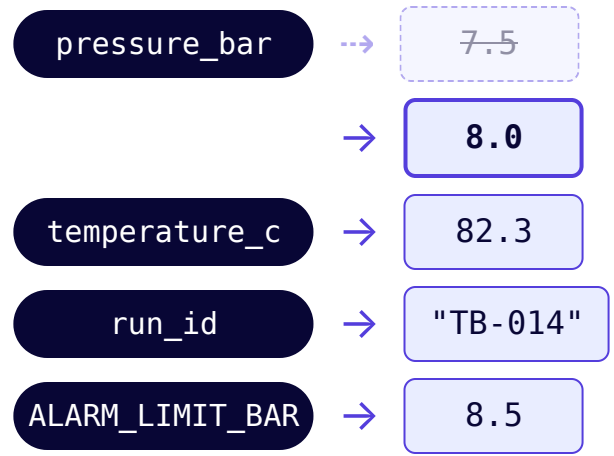


Variables are names for values

```
pressure_bar = 7.5
temperature_c = 82.3
run_id = "TB-014"
ALARM_LIMIT_BAR = 8.5

pressure_bar = 8.0
print(pressure_bar)
print(temperature_c)
```

```
8.0
82.3
```



`=` is **assignment**: it gives a value a name. By convention: `snake_case` for variables, `CAPITALS` for **constants**.

Question After `pressure_bar = 8.0`, what happens to the bytes that held 7.5?

Every value has a type

Why The type decides what an operation does: `7 + 5` is 12, `"7" + "5"` is "75".

```
run_id = "TB-014"        # str: text, a string
cycles = 12              # int: whole number
flow_l_s = 18.4          # float: decimal number
is_stable = True         # bool: True or False
operator = None          # None: no value (yet)

print(type(flow_l_s))
```

```
<class 'float'>
```

Text that looks like a number is still text

```
text = "7.5"
pressure_bar = float(text)
print(pressure_bar + 1)        # 8.5

count = 3
print("Cycles: " + count)
```

```
TypeError: can only concatenate str (not "int") to str
```

> `float(text)` **converts** text to a number. Python never converts for you.

Question `"7.5" * 2` gives no error. What does it print?



f-strings put values inside text

Why Every result you report is text with numbers in it: a log line, a file name, a plot title. An f-string sets the decimals and the unit in one place.

```
run_id = "TB-014"
pressure_bar = 7.5
print(f"Run {run_id}: {pressure_bar:.1f} bar")
print(f"{pressure_bar * 14.5038:.0f} psi")
print("TB" in run_id, run_id.lower())
print("12.5 kW".split())
```

```
Run TB-014: 7.5 bar
109 psi
True tb-014
['12.5', 'kW']
```

> A **method** is a function you call on a value with a dot: `run_id.lower()`



A comparison gives a boolean: True or False

```
pressure_bar = 8.7
temperature_c = 82
is_alarm = pressure_bar > 8.5
print(is_alarm, type(is_alarm))
print(pressure_bar > 8.5 and temperature_c > 90)
print(pressure_bar > 8.5 or temperature_c > 90)
print(not is_alarm)
print(7.0 < pressure_bar < 9.0)
```

```
True <class 'bool'>
False
True
False
True
```

Compare with `>` `<` `>=` `<=` `==` `!=`. Combine with **and**, **or**, **not**. One `=` assigns, two `==` compare.



A number has no unit, so put it in the name

Why Python stores 7.5, not 7.5 bar. Mix bar and psi and you get a wrong number, not an error.

Written fast

```
p = 7.5
t = 82
calc(p, t)
```

Read easily

```
pressure_bar = 7.5
temperature_c = 82
to_kelvin(temperature_c)
```

Put the unit in the name: `ALARM_LIMIT_BAR = 8.5`

A traceback is a map to the error

```
Traceback (most recent call last):
  File "convert.py", line 2, in <module>
    pressure_bar = float(text)
ValueError: could not convert string to float: 'seven'
```

- Line 1 is only a header, it says: read on
- The middle shows the file, the line number and the code that failed
- The last line names the error type and what went wrong. Read it first.



Exercise 02: Variables and types

`exercises/python/02-variables-and-types`

Words you can now use variable · assignment · type · string · float · f-string · convert · traceback · boolean

A list keeps values in order

```
pressures_bar = [6.8, 7.1, 7.5, 7.4, 7.9]
print(pressures_bar[0])
print(len(pressures_bar))
pressures_bar.append(8.2)
print(max(pressures_bar))
print(sorted(pressures_bar, reverse=True))
```

```
6.8
5
8.2
[8.2, 7.9, 7.5, 7.4, 7.1, 6.8]
```




Indexing starts at zero

index	0	1	2	3	4
value	6.8	7.1	7.5	7.4	7.9
negative	-5	-4	-3	-2	-1

```
p = [6.8, 7.1, 7.5, 7.4, 7.9]
print(p[1:3], p[:2], p[-2:])
```

```
[7.1, 7.5] [6.8, 7.1] [7.4, 7.9]
```

A **slice** such as `p[1:3]` takes a part of the list. The end index is not included.

Question `p[10]` and `p[3:10]`: which one fails?



A dict maps keys to values

```
reading = {"run": "TB-014", "pressure_bar": 7.5}
print(reading["pressure_bar"])
reading["flow_l_s"] = 18.4
print(reading.get("power_kw"))
print(reading)
```

```
7.5
None
{'run': 'TB-014', 'pressure_bar': 7.5, 'flow_l_s': 18.4}
```

Each entry is a **key** with its **value**: `"pressure_bar"` is a key, `7.5` is its value.

Question A dict with a million keys finds one as fast as a dict with ten. How?



A tuple is a fixed group of values

```
position = (52.01, 4.36)
latitude, longitude = position
print(latitude)
position[0] = 0.0
```

```
52.01
TypeError: 'tuple' object does not support item assignment
```

- A **tuple** is written with round brackets and cannot be changed after it is made
- **Unpacking** gives each value its own name. A function that returns two values returns a tuple
- A **set** keeps only unique values: `set(["P1", "T1", "P1"])` is `{"P1", "T1"}`



Four collections, picked by how you find a value

	list	tuple	dict	set
Find by	Position, in order	Position, in order	Key	Is it in there?
Change it?	Yes	No	Yes	Yes, no duplicates
Fits	A series of the same kind of value	A fixed group that belongs together	One record with named fields	Unique values, membership
Example	`pressures[2]`	`lat, lon = pos`	`reading["run"]`	`"P1" in ids`

> Copilot will pick one for you unless you say which you want.



Exercise 03: Collections

`exercises/python/03-collections`

Words you can now use list · tuple · index · slice · dict · key · value · set · method



if runs the first branch that is true

```
pressure_bar = 9.4
if pressure_bar > 9.0:
    print("ALARM: over limit")
elif pressure_bar < 5.0:
    print("Warning: low pressure")
else:
    print("OK")
```

```
ALARM: over limit
```

`pressure_bar > 9.0`

true: run this branch, skip the rest

`pressure_bar < 5.0`

not checked

`else`

not checked

Checked top to bottom, the first true one wins

The **condition** uses a **comparison operator**: `>` `<` `>=` `==`. **Indentation** of 4 spaces marks the block.

`for` runs the block once per element

```
pressures_bar = [6.8, 7.1, 9.4]
for p in pressures_bar:
    print(f"{p:.1f} bar")

for i in range(3):
    print(i)
```

```
6.8 bar
7.1 bar
9.4 bar
0
1
2
```

To **iterate** is to go through a collection one element at a time. With the position: `for n, p in`



Most scripts initialise, loop, test, collect

```
pressures_bar = [6.8, 7.1, 9.4, 7.4, 9.1]
high = []
total = 0.0

for p in pressures_bar:
    total += p
    if p > 9.0:
        high.append(p)

print(high, round(total / len(pressures_bar), 2))
```

```
[9.4, 9.1] 7.96
```

1. **INITIALISE**
   An empty list and a zero total
2. **LOOP**
   Take each value in turn
3. **TEST**
   Decide with an `if`
4. **COLLECT**
   Report at the end



A list comprehension is that loop on one line

The loop

```
high = []
for p in pressures_bar:
    if p > 9.0:
        high.append(p)
```

The comprehension

```
high = [p for p in pressures_bar if p > 9.0]
```

```
# transform every value
psi = [p * 14.5038 for p in pressures_bar]

# the same idea without the brackets
total_high = sum(p for p in pressures_bar if p > 9.0)
```

One `for`, one `if`. Anything more, write the loop.



Exercise 04: Control flow

`exercises/python/04-control-flow`

Words you can now use condition · branch · loop · iterate · block · indentation · comprehension



A function names a piece of logic

```
def to_psi(pressure_bar):
    return round(pressure_bar * 14.5038, 1)

print(to_psi(7.5))
print(to_psi(10))
```

```
108.8
145.0
```



return passes a value on, print only shows it

```
def double_and_print(x):
    print(x * 2)

result = double_and_print(4)
print(result)
```

```
8
None
```

A function without `return` gives back `None`. Use `print` to show, `return` to pass on.



A script is small functions plus a main

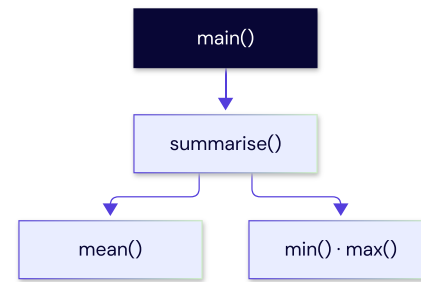
Why Small functions can be tested and reused one by one. `main` shows the order of the work in a few lines.

```
def mean(values):
    """Return the arithmetic mean of a list of numbers."""
    return sum(values) / len(values)

def summarise(values):
    return {"min": min(values), "max": max(values), "mean": mean(values)}

def main():
    pressures_bar = [6.8, 7.1, 9.4, 7.4]
    print(summarise(pressures_bar))

if __name__ == "__main__":
    main()
```



> The text in triple quotes is a **docstring**: it says what the function does.

Type hints tell you how to call a function

```
def to_psi(pressure_bar: float, decimals: int = 1) -> float:
    return round(pressure_bar * 14.5038, decimals)
```

pressure_bar: float

The parameter and the type it expects

decimals: int = 1

A type and a default value, so you may leave it out

-> float

The type of the value that comes back

Hints are documentation that the editor and tools can check. Python does not enforce them when the script runs.

Question The hint says `float`. What happens with `to_psi("7.5")`?

import gives you ready-made tools like math

```
import math
from math import sqrt

diameter_m = 0.05
area_m2 = math.pi * (diameter_m / 2) ** 2
flow_m3_s = 18.4 / 1000
print(round(flow_m3_s / area_m2, 2))

print(math.isclose(0.1 + 0.2, 0.3), sqrt(16))
```

```
9.37
True 4.0
```




Move code outward only when you need to

	Use it when	The cost
Lines in the script	You do it once, in a few lines	Copy and paste when you need it again
A function	You repeat it, or it deserves a name	A little structure to read through
A module	Several scripts need the same functions	One more file, and an `import`

> Copilot will pick one for you unless you say which you want.



A test says what correct means

```python
import pytest
import conversions

def test_bar_to_psi():
    assert conversions.bar_to_psi(1) == pytest.approx(14.5038)
```

Red: run, read the failure → Green: make it pass → Test first: write the next test

> This loop is **test-driven development**.

> `assert` states an **assertion**: a claim that must be true, or the test fails.



Exercise 05: Functions

`exercises/python/05-functions`

Words you can now use function · argument · return value · type hint · test · assertion · test-driven development · import · module

Files and the standard library



pathlib builds paths and finds files

```
from pathlib import Path

data_dir = Path("data")
log_file = data_dir / "run_014.csv"

print(log_file.name, log_file.suffix, log_file.stem)
print(log_file.exists())

for f in sorted(data_dir.glob("*.csv")):
    print(f)
```

```
run_014.csv .csv run_014
True
data\run_014.csv
data\run_015.csv
```




with open reads a file and closes it again

```
from pathlib import Path

with open("data/run_014.csv", encoding="utf-8") as f:
    for line in f:
        print(line.strip())

text = Path("data/run_014.csv").read_text(encoding="utf-8")
```

```
t_s,pressure_bar,temperature_c
0,6.8,71.2
10,7.1,74.5
20,7.5,78.9
```

`f` is the **file handle**. State the **encoding**, or Windows picks its own.



Mode `w` overwrites, mode `a` appends

```
lines = ["run,max_pressure_bar", "TB-014,7.9"]
with open("summary.txt", "w", encoding="utf-8") as f:
    for line in lines:
        f.write(line + "\n")

with open("summary.txt", "a", encoding="utf-8") as f:
    f.write("TB-015,8.2\n")
```



Mode `"w"` replaces an existing file without asking. `"a"` adds at the end, `"r"` only reads.

`csv` turns each row into a dict of strings

```
t_s,pressure_bar,temperature_c
0,6.8,71.2
10,7.1,74.5
20,7.5,78.9
```

```
import csv
with open("data/run_014.csv", newline="", encoding="utf-8") as f:
    rows = list(csv.DictReader(f))   # the header row gives the keys
print(rows[0])
print(float(rows[1]["pressure_bar"]) + 1)
```

```
{'t_s': '0', 'pressure_bar': '6.8', 'temperature_c': '71.2'}
8.1
```

Question The log has ten million rows. What does `list(...)` on line 3 do?



json stores settings and nested data

Why CSV holds one flat table. Settings, limits and nested results need a format with names and levels.

```
import json
config = {"limit_bar": 9.0, "channels": ["pressure", "flow"], "operator": None}
text = json.dumps(config)
print(text)
back = json.loads(text)
print(back["channels"][0], back["operator"])
```

```
{"limit_bar": 9.0, "channels": ["pressure", "flow"], "operator": null}
pressure None
```




Exercise 06: Standard library

`exercises/python/06-standard-library`

Words you can now use standard library · path · glob · file handle · encoding · CSV · JSON



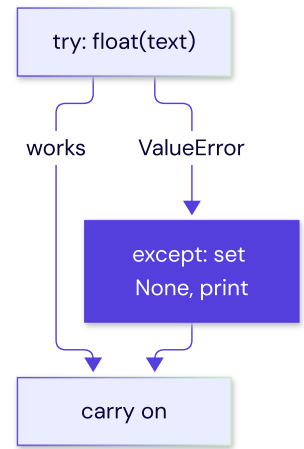
Exceptions



try / except handles an error you expect

```
text = "seven"
try:
    pressure_bar = float(text)
except ValueError:
    pressure_bar = None
    print(f"Skipping bad value: {text!r}")
print(pressure_bar)
```

```
Skipping bad value: 'seven'
None
```



`float("seven")` **raises a** `ValueError`**.** `except` **catches it.**

Question What changes if you write `except:` without `ValueError`?

No plan for an error? Let it bubble up

Swallowed

```
try:
    mean_bar = summarise(path)
except Exception:
    pass
```

The script carries on. The report is wrong or empty, and nothing tells you why.

Bubbles up

```
mean_bar = summarise(path)
```

The error travels up through every function that called this one. Nobody catches it, so the script stops with a traceback: file, line and reason.

- Catch an error only where you can do something with it: skip the row and count it, use a default, ask again
- Need to add context? Catch it, then `raise` again with a better message
- A script that stops with a traceback is better than a script that finishes with a wrong number



Exercise 07: Files and exceptions

`exercises/python/07-files-and-exceptions`

Words you can now use exception · raise · catch · bubble up



Projects with uv



Python packaging has a messy history



2007

virtualenv

a separate set of packages per project



2008

pip

installs packages, and nothing else



2012

conda

packages and Python itself, in its own world



2017

pipenv

pip and virtualenv in one, with a lockfile



2018

Poetry

one project file, a lockfile, publishing



2024

uv

all of it in one fast tool

- Each tool fixed one piece: packages, environments, Python versions, locking
- Also in this crowd: `pyenv`, `pip-tools`, Hatch, PDM, Rye
- What they agreed on in the end is one project file: `pyproject.toml`
- You will meet all of these in tutorials, old projects and Copilot answers

A virtual environment stops projects breaking each other

One shared set of packages

Python on your laptop

~~numpy 1.26~~ numpy 2.5

bench-report
written for numpy 1.26

breaks

vibration-analysis
needs numpy 2.5

works

Installing for one project replaces the package for all of them

A .venv per project

bench-report

`.venv`
numpy 1.26

works

vibration-analysis

`.venv`
numpy 2.5

works

Each project folder holds its own packages, and its own Python version

`.venv` is only a folder. Delete it and `uv sync` builds it again from the project file, so it never goes into Git.

uv and pip: the same job, fewer steps

	pip workflow	uv workflow
Environment	`python -m venv .venv`	`uv init`, then automatic
Activate	`.venv\Scripts\Activate.ps1`	Not needed: `uv run`
Install	`pip install matplotlib`	`uv add matplotlib`
Declare	`requirements.txt`, by hand	`pyproject.toml`, updated by uv
Pin versions	`pip freeze`, if you remember	`uv.lock`, automatic



Two commands give a project its own packages

```
uv init --no-package bench-tools
cd bench-tools
uv add matplotlib
uv run python main.py
```

```
Creating virtual environment at: .venv
Resolved 12 packages in 330ms
Installed 11 packages in 68ms
 + matplotlib==3.11.2
 + numpy==2.5.4
 ...
```

> numpy came along without you asking, because matplotlib needs it: a **transitive dependency**.



pyproject.toml declares, uv.lock pins

```toml
[project]
name = "bench-tools"
requires-python = ">=3.13"
dependencies = [
    "matplotlib>=3.11.2",
]
```

Commit

`pyproject.toml`, `uv.lock`, `.python-version` and your `.py` files

Do not commit

`.venv/`, big and machine specific

Question The script ran in March and fails in November. Nobody touched the code. What changed?



One script, no project: the packages go in the file

```python
# /// script
# requires-python = ">=3.13"
# dependencies = [
#     "matplotlib>=3.11.2",
# ]
# ///
import matplotlib.pyplot as plt

plt.plot([7.1, 7.4, 7.6])
plt.savefig("pressure.png")
```

```
uv add --script plot.py matplotlib
uv run plot.py
```

- The comment block at the top is **inline script metadata**, a Python standard (PEP 723)
- `uv add --script` writes it for you
- `uv run` reads it, builds a temporary environment and runs the script

One file to mail to a colleague: use this. Several files, tests or a lockfile: make a project.



Exercise 08: A uv project

`exercises/python/08-uv-project`

Words you can now use virtual environment · package · dependency · lockfile · pin · sync



Python goes further, but not today

- **Styles**: procedural (today), object-oriented with classes and dataclasses, functional
- **Language**: decorators, generators, context managers
- **Debugging**: the VS Code debugger, with breakpoints and stepping, instead of `print`
- **Scripts that grow**: `logging` instead of `print`, arguments with `argparse`

- **Performance**: measure first (`timeit`, a profiler), then NumPy
- **Concurrency**: threads, processes, `asyncio`
- **Extending**: C, or Rust through PyO3
- **Packaging**: your own code, installable by colleagues



Questions?

The Python part



Version control



This prompt will make sense by the end of today

```
Fetch and rebase `feature/max-temperature` onto `origin/main`. If a commit
conflicts, stop and show me both sides. Then stage only `analyse_run.py`
and commit it as a conventional commit, type `fix`, scope `analysis`. Do
not commit `output/`. Push with `--force-with-lease` and draft the pull
request description from `git diff origin/main`.
```




final_v3 is version control by hand

```
\\fileserver\compressors\testbench\
    analyse_run.py
    analyse_run_final.py
    analyse_run_final_v3_USE_THIS_ONE.py
```

> Git answers these four questions without copies of the file.

History

Which version made last week's report?

Who and why

Who changed the conversion factor?

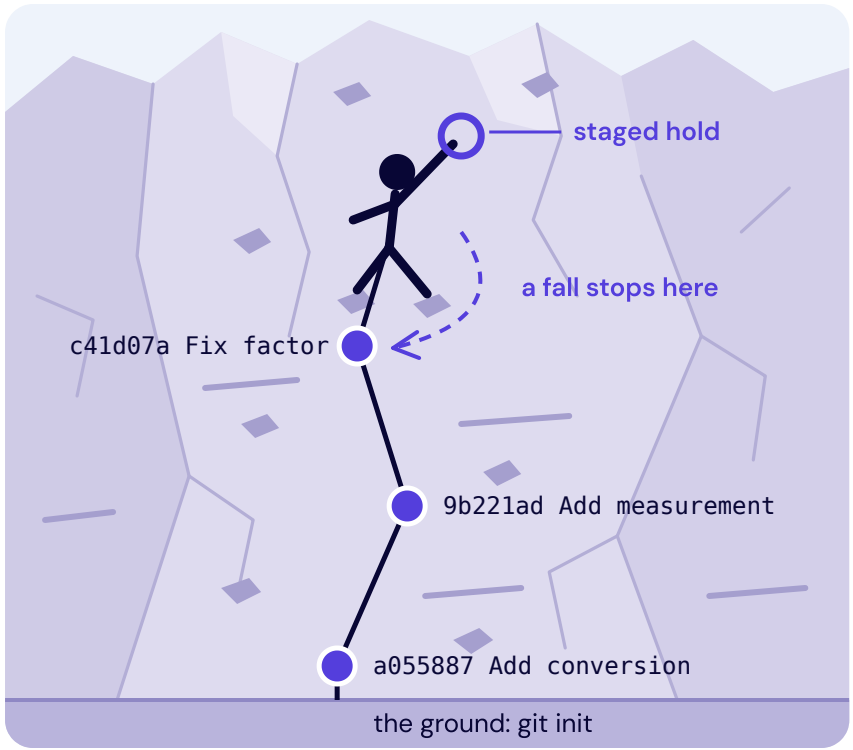
Undo

How do I get back to what worked?

Collaborate

Two people edit, whose version wins?

Stage a hold, then hammer in an anchor



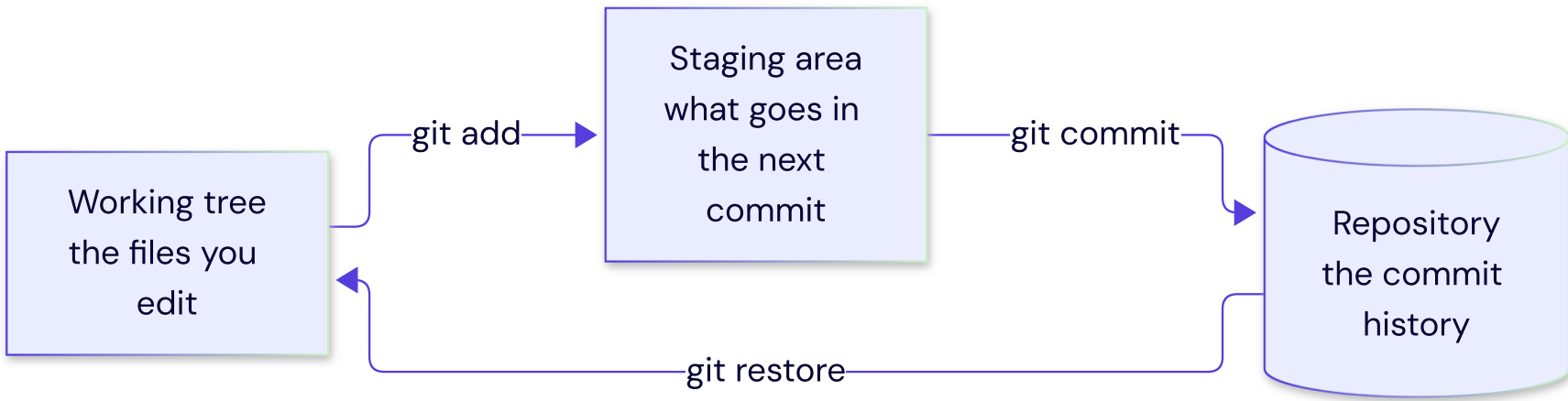
Working tree
You try moves above the last anchor. Nothing is saved yet.

Stage `git add`
You pick the holds for the next move.

Commit `git commit`
You hammer in an anchor and clip the rope. A fall takes you back to it, not to the ground.

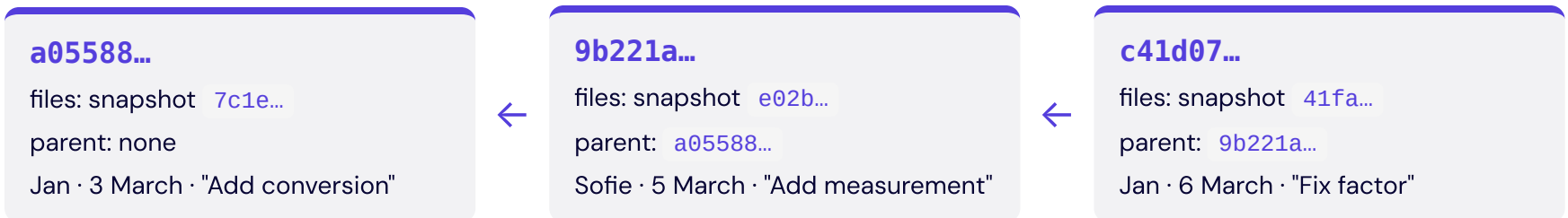
History
The line of anchors below you.

A change moves through three areas



- You choose what goes into a commit by **staging** it first (the staging area is also called the **index**)
- `git status` tells you which area each change is in

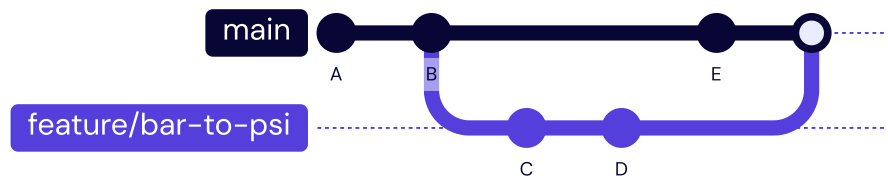
A hash is the name Git gives to content



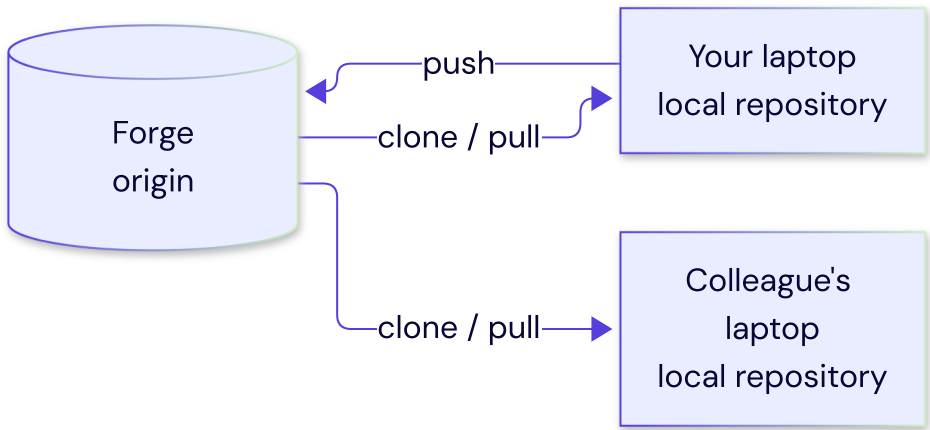
- Git stores every file and every commit under the **hash of its content**: 40 characters, calculated with SHA-1
- Same content, same hash. Change one character and the hash changes
- A commit contains its parent's hash, so changing an old commit changes every hash after it
- The first 7 characters are enough to point at a commit: `git show c41d07a`

A branch is a movable label on a commit

- **Repository**: a project folder plus its full history
- **Commit**: a saved snapshot with a message, author and date
- **Branch**: a movable label on a commit
- Work on a branch does not touch `main` until you merge it



A forge hosts the shared remote



- **Git** is the free tool on your laptop, a **forge** is the website that hosts the shared copy
- A **remote** is that shared copy, by convention called `origin`

Words you can now use repository · commit · branch · remote · working tree · staging area

Git on the command line



Use Git from the command line or VS Code

VS Code

Easier to use.

- Changed files, diffs and conflicts side by side
- Stage, commit and push with a click
- Next to the code you are editing

Command line

A skill that carries over.

- Works the same on a server, in a pipeline, on any laptop
- Every feature, and every error message and tutorial uses it
- Shows what Git does, step by step

An agent drives Git through commands. You can only judge what it did, or is about to do, if you can read them. So today: the command line first.



Set your identity once, then init a repository

```
git config --global user.name "Your Name"
git config --global user.email "you@atlascopco.com"
mkdir testbench
cd testbench
git init -b main
git status
```

- Identity: once per laptop, same email as your account on the forge
- `git status` never changes anything: run it all the time

> A new file is **untracked**: Git sees it, but does not follow it until you `git add` it.

Stage what belongs together, then commit

```
git add conversions.py
git commit -m "feat(conversions): add psi to bar conversion"
```

`type(scope): description` is a **conventional commit**. The scope is optional.

Type	Use it for	Example
`feat`	new behaviour	`feat: report max temperature`
`fix`	a bug fix	`fix(conversions): correct psi to bar factor`
`docs` `test` `refactor` `chore`	no change in behaviour	`docs: explain the csv columns`

- Description: short, imperative, one logical change. "and" in it means two commits
- `git add .` stages everything: check `git status` first



log and diff show what changed

```
git log --oneline
git log --oneline --graph --all
```

```
9b221ad chore(data): add first measurement
a055887 feat(conversions): add psi to bar conversion
```

```
git diff            # edits not yet staged
git diff --staged   # what the next commit will contain
```

- `--graph --all` draws every branch as lines: you will read it in the branches exercise
- If the output is long, Git opens a pager: **q** leaves it



Exercise time: first commit

`exercises/git/01-first-commit`

```
uv run python exercises/git/setup_exercise.py 1
```



show and restore bring back old versions

```
git show a055887                    # one commit: message and diff
git show HEAD~1:conversions.py      # a file as it was one commit ago
git restore --source=HEAD~1 conversions.py
git restore conversions.py          # discard edits since last commit
```

- `HEAD` is the latest commit, `HEAD~1` the one before it
- `git restore <file>` throws your edits away: there is no undo for that



Extra exercise: history

`exercises/git/02-history`

When you are ahead, or after class: `uv run python exercises/git/setup_exercise.py 2`



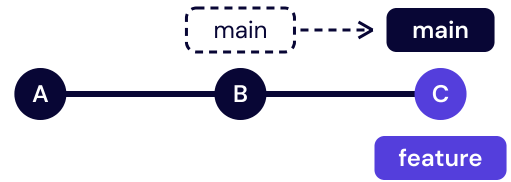
Work on a branch, then merge it into main

```
git switch -c feature/bar-to-psi
git add conversions.py
git commit -m "Add bar to psi conversion"
git switch main
git merge feature/bar-to-psi
```

```
Updating 9b221ad..1cd9279
Fast-forward
```

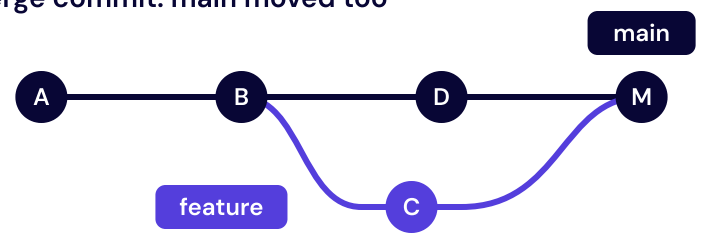
- `git branch` lists branches
- `git branch -d <name>` deletes one

Fast-forward: main did not move



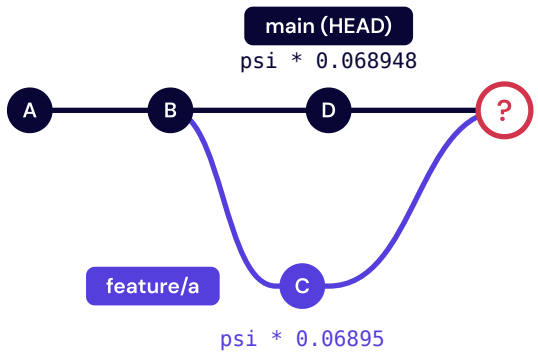
the label moves,
no new commit

Merge commit: main moved too



M has two parents: D and C

A conflict means Git needs you to decide



Both changed the same line after B.

```
CONFLICT (content): Merge conflict in conversions.py

def psi_to_bar(psi):
<<<<<<< HEAD
    return psi * 0.068948
=======
    return psi * 0.06895
>>>>>>> feature/a
```

Above `=======` is your branch. **Below** is the incoming branch.

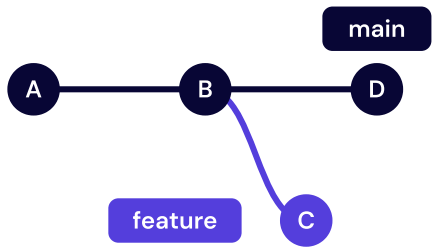
- Nothing is lost. Edit the file, delete the marker lines, then `git add` and `git commit`
- Changed your mind? `git merge --abort`
- **Tip:** agents are excellent at solving merge conflicts. Read the result with `git diff` before you commit

Question Two people change different lines of the same file. Is that a conflict?



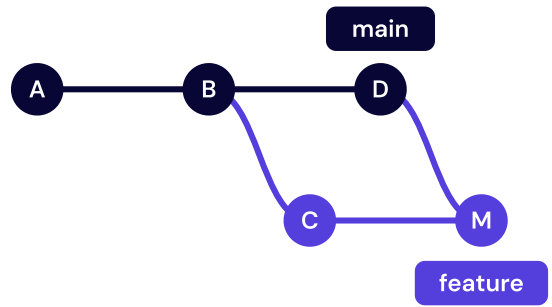
Rebase replays your commits on top of main

Before: main moved on



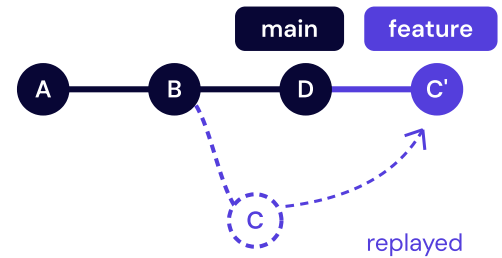
C was made on top of B.

git merge main



Both lines stay, plus a merge commit M.

git rebase main



One straight line. C' has a new hash.

```
git switch feature/x; git rebase main
git add <file>; git rebase --continue   # after fixing a conflict
git rebase --abort                      # give up
```

> Rebase before you push. Rebasing rewrites commits, so a branch you already pushed needs `git push --force-with-lease` afterwards, and colleagues who pulled the old commits get duplicates.

Exercise time: branches

`exercises/git/03-branches`

```
uv run python exercises/git/setup_exercise.py 3
```

Words you can now use stage · commit message · diff · HEAD · merge · fast-forward · conflict · rebase



Remotes and pull requests



Clone copies the repository and its history

```
git clone https://<forge>/<organisation>/<training-repo>.git
cd <training-repo>
git remote -v
```

```
origin  https://<forge>/<organisation>/<training-repo>.git (fetch)
origin  https://<forge>/<organisation>/<training-repo>.git (push)
```

- The first time, a browser window asks you to sign in (Git Credential Manager)
- `clone` creates the remote `origin`; the URL is on the repository page of the forge

Push uploads commits, pull downloads them

```
git push -u origin feature/bar-to-psi   # first push of a new branch
git push                                 # later pushes
git pull                                 # fetch and merge new commits
git fetch                                # only download, change nothing
```

- `-u` sets the upstream: it links your local branch to the remote one, needed only once
- `origin/main` is a **remote-tracking branch**: your laptop's last known copy of `main` on the server, updated by `fetch` and `pull`
- `pull` is `fetch` plus merge: commit or restore your edits first

Question Your laptop is stolen on the train. Which work is gone?



Exercise time: the remote

`exercises/git/04-remote`

Clone, branch, commit, push



A pull request lets a colleague review first

Push → Open → Review → Approve → Merge

Propose

A pull request (PR) proposes merging your branch into `main`. Colleagues read the diff first.

Merge

Click **Merge pull request**, then **Delete branch**.

Update your laptop

`git switch main`, then `git pull`

A **branch protection rule** on `main` can require a pull request with an approval before it can be merged.

Exercise time: pull request

`exercises/git/05-pull-request`

PR into main, review, merge, pull



.gitignore keeps files out of history

```
# .gitignore
.venv/
__pycache__/
data/raw/
```

- One pattern per line; commit the file itself so everyone shares it
- Never commit passwords, tokens or large raw data
- Day 2 uses it for data files and notebooks

> A **tracked** file is one Git already follows. `.gitignore` only works on files that are not tracked yet: they become **ignored**.



Demo: Git in VS Code

Optional practice: `exercises/git/06-git-in-vscode`

In the **Source Control view**: + stages, Commit commits, **Publish Branch** is the first push, **Sync Changes** pulls and then pushes.

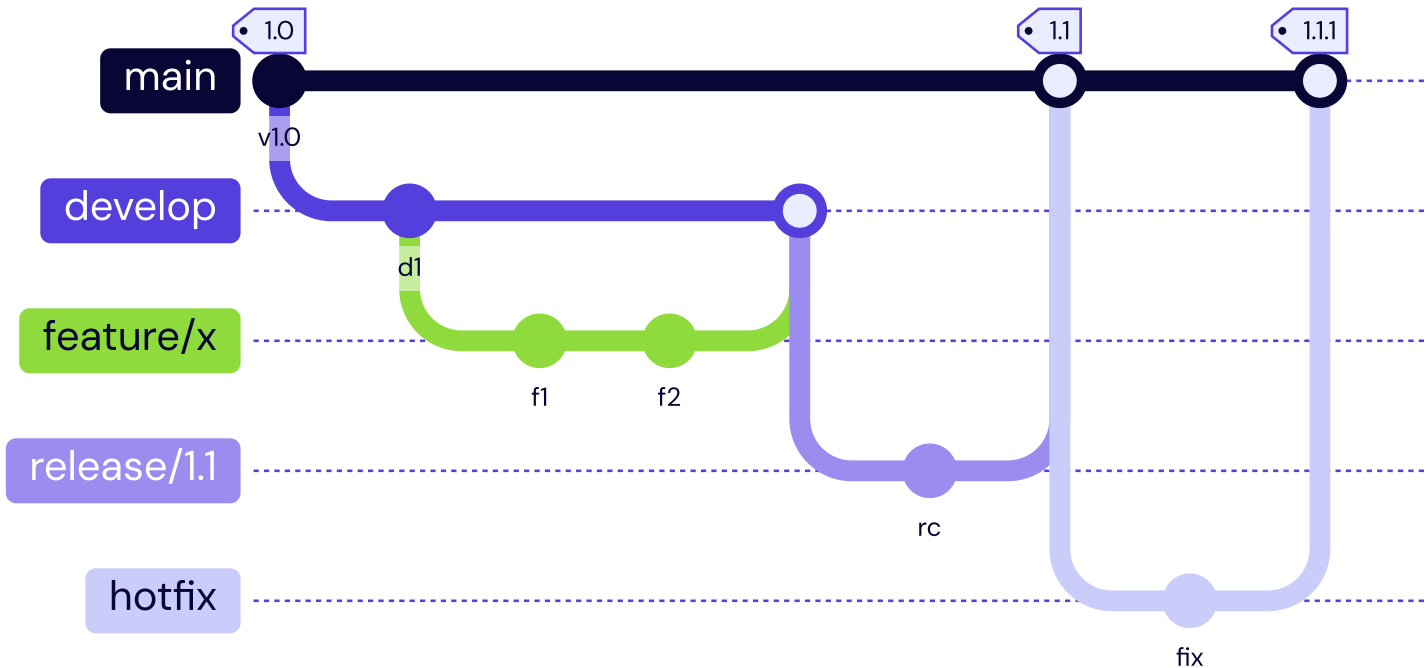
Words you can now use clone · origin · push · pull · fetch · upstream · origin/main · pull request · review



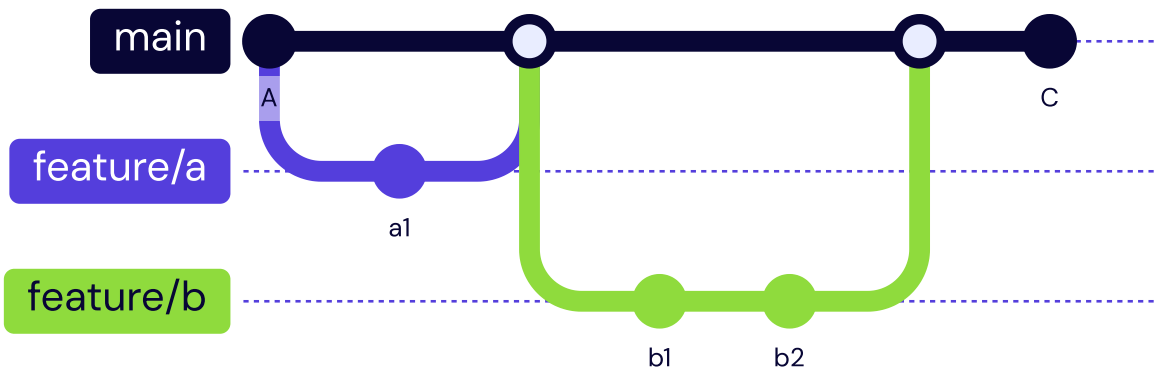
Branching models



GitFlow keeps a branch per release stage



Trunk-based: one main, short branches



- One long-lived branch, `main`; short branches merged through a pull request

Use trunk-based for scripts and analyses

	GitFlow	Trunk-based
Fits	Versioned products, several releases in support	Continuous delivery, scripts, analyses, small teams
Cost	Many branches, large merges, more conflicts	Discipline: small changes, review, tests

- **Use trunk-based**: short-lived `feature/...` branches and PRs, enforced by a branch protection rule on `main`
- Exists, not today: `git cherry-pick`

Words you can now use trunk-based · GitFlow · feature branch · short-lived · branch protection rule



Questions?

The Git part



You can now do all six things

Run a script

From PowerShell and VS Code

Manage a project

A .venv per project with uv

Write a program

Functions, loops, imports

Read an error

Find the line in a traceback

Automate files

Read files, write a summary

Work with Git

Branch, commit, push, PR, pull

Today: the **vocabulary** and the tradeoffs. Day 3: you use them to direct and review Copilot and coding agents.

Read these two prompts again

Git

```
Fetch and rebase `feature/max-temperature` onto `origin/main`. If a commit conflicts, stop and show me both sides. Then stage only `analyse_run.py` and commit it as a conventional commit, type `fix`, scope `analysis`. Do not commit `output/`. Push with `--force-with-lease` and draft the pull request description from `git diff origin/main`.
```

Python

```
Write a function `summarize_run(path: Path) -> dict` that reads a CSV log with `csv.DictReader`, converts `pressure_bar` to float, skips and counts rows that raise `ValueError`, and returns the mean and max. Use a list comprehension where it fits. Standard library only, no classes. Add a pytest test using `pytest.approx`.
```




Eleven commands cover daily Git

Local

```
git status
git add <file>
git commit -m "message"
git log --oneline
git diff
git restore <file>
git switch -c <branch>
git merge <branch>
```

Remote

```
git clone <url>
git switch main
git pull
git switch -c feature/x
git push -u origin feature/x
# open the PR, merge it, repeat
```

Before day 2: one full cycle with a PR

Before day 2

- A working uv environment
- Redo the fundamentals you found hard
- One full clone, commit, push, pull cycle with a PR
- **Bring your own work**: a use case, an example or a question

Read more

- Pro Git book (`git-scm.com/book`)
- Learn Git Branching (`learngitbranching.js.org`)
- Python tutorial (`docs.python.org/3/tutorial`)
- uv docs (`docs.astral.sh/uv`)

Next days

- Day 2: pandas, NumPy, SciPy, Matplotlib, scikit-learn
- Day 3, going deeper: tests, linting, pipelines, coding agents and reviewing their work



More from Dataminded Academy



Containerization with Docker

Pack a script with its Python and packages, so it runs the same anywhere

`github.com/datamindedacademy/containerization_with_docker`



Orchestration with Airflow

Run scripts on a schedule, in order, with retries and alerts

`github.com/datamindedacademy/workflows_with_airflow`



Polars and PySpark

Data frames for large logs: Polars on one machine, PySpark on a cluster

`github.com/datamindedacademy/dataframe-of-mind`





Thank you

See you on day 2



Backup: more Python



Same values passed to every function? Use a class

```python
class Compressor:
    def __init__(self, name, limit_bar):     # runs when you create one
        self.name = name
        self.limit_bar = limit_bar
        self.pressures_bar = []

    def add(self, pressure_bar):             # a method: a function of the object
        self.pressures_bar.append(pressure_bar)

    def is_over_limit(self):
        return max(self.pressures_bar) > self.limit_bar

c101 = Compressor("C-101", limit_bar=9.0)    # an object, or instance
c101.add(9.4)
print(c101.name, c101.is_over_limit())       # C-101 True
```

You used objects all day: `readings.append(7.5)` calls a method of a list. A class lets you define your own type.



A dataclass catches the typo a dict lets through

```python
from dataclasses import dataclass

@dataclass
class Reading:
    t_s: int
    pressure_bar: float
    temperature_c: float

reading = Reading(t_s=10, pressure_bar=7.1, temperature_c=74.5)
print(reading)                              # Reading(t_s=10, pressure_bar=7.1, temperature_c=74.5)
print(reading.pressure_bar)                 # 7.1, and the editor completes the name
print(reading == Reading(10, 7.1, 74.5))    # True
```

A dict accepts any key, so a typo gives a new key. A dataclass has fixed fields, so a typo is an error.

Question The field says `float`. What does `Reading(10, "7.1", 74.5)` give?

pydantic rejects bad data before it spreads

```python
from pydantic import BaseModel, Field

class Reading(BaseModel):
    t_s: int
    pressure_bar: float = Field(gt=0, lt=20)   # a rule: between 0 and 20
    temperature_c: float

row = {"t_s": "10", "pressure_bar": "7.1", "temperature_c": "74.5"}   # from csv.DictReader
Reading(**row)        # Reading(t_s=10, pressure_bar=7.1, temperature_c=74.5)

Reading(t_s=20, pressure_bar=-1, temperature_c="hot")
```

```
ValidationError: 2 validation errors for Reading
pressure_bar
  Input should be greater than 0 [type=greater_than, input_value=-1, input_type=int]
temperature_c
  Input should be a valid number, unable to parse string as a number [input_value='hot']
```

Not in the standard library: `uv add pydantic`. Validate at the edge (files, APIs, user input), then trust the data inside.



A generator reads a file too big for memory

A list: everything in memory

```
def read_pressures(path):
    pressures = []
    with open(path, newline="") as f:
        for row in csv.DictReader(f):
            pressures.append(
                float(row["pressure_bar"]))
    return pressures
```

A generator: one row at a time

```
def read_pressures(path):
    with open(path, newline="") as f:
        for row in csv.DictReader(f):
            yield float(row["pressure_bar"])
```

```
print(max(read_pressures("data/run_014.csv")))    # 7.5, also for ten million rows
for pressure_bar in read_pressures("data/run_014.csv"):
    ...
```

A generator runs **once**. Loop over it a second time and it is empty. Need the values twice? Make a list.



Time or cache any function with a decorator

Using one

```
@timed
def summarize_run(path):
    return max(read_pressures(path))

summarize_run("data/run_014.csv")
```

```
summarize_run: 0.42 s
```

The `@` line means:

`summarize_run = timed(summarize_run)`

Writing one

```
import functools, time

def timed(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        took = time.perf_counter() - start
        print(f"{func.__name__}: {took:.2f} s")
        return result
    return wrapper
```

You will mostly **use** them: `@dataclass`, `@functools.cache`, `@pytest.fixture`, `@app.get("/runs")`.



A context manager never forgets to clean up

with, as you know it

```
with open(path) as f:
    text = f.read()
# the file is closed here,
# also when read() raised
```

The same pattern for a database connection, a lock, a temporary folder, a serial port

Your own

```
import time
from contextlib import contextmanager

@contextmanager
def timer(label):
    start = time.perf_counter()  # set up
    try:
        yield                    # the block runs
    finally:                     # clean up
        took = time.perf_counter() - start
        print(f"{label}: {took:.2f} s")

with timer("read logs"):
    rows = read_all_logs()
```

`try / finally` is the long form: the `finally` block runs whether the code above it failed or not.



asyncio waits for many slow things at once

One after the other: 3 s

```
import time

def read_pressure(name):
    time.sleep(1)    # a slow network request
    return name, 7.5

def main():
    names = ["C-101", "C-102", "C-103"]
    readings = [read_pressure(name)
                for name in names]
    print(readings)

main()
```

All at once: 1 s

```
import asyncio

async def read_pressure(name):
    await asyncio.sleep(1)
    return name, 7.5

async def main():
    names = ["C-101", "C-102", "C-103"]
    readings = await asyncio.gather(
        *(read_pressure(name) for name in names))
    print(readings)

asyncio.run(main())
```

It speeds up **waiting** (network, databases, instruments), not **calculating**. For heavy maths: NumPy, or several processes.



logging shows what an unattended script did

```python
import logging

logging.basicConfig(
    level=logging.INFO,                                  # the switch: show INFO and above
    format="%(asctime)s %(levelname)s %(message)s",
)
log = logging.getLogger(__name__)

log.debug("Row %d: %s", number, row)                     # hidden at level INFO
log.info("Read %d rows from %s", len(rows), path)
log.warning("Skipped %d rows without a pressure", skipped)
```

```
2025-03-05 14:30:02,118 INFO Read 3 rows from data/run_014.csv
2025-03-05 14:30:02,118 WARNING Skipped 2 rows without a pressure
```

Five levels: `DEBUG`, `INFO`, `WARNING`, `ERROR`, `CRITICAL`. Add `filename="run.log"` to `basicConfig` to write to a file.



argparse changes the input without editing the script

```python
import argparse
from pathlib import Path

parser = argparse.ArgumentParser(description="Summarise one test bench run.")
parser.add_argument("log", type=Path, help="CSV file of the run")
parser.add_argument("--limit-bar", type=float, default=9.0, help="alarm limit")
args = parser.parse_args()

print(args.log, args.limit_bar)
```

```
uv run python summarize_run.py data\run_014.csv --limit-bar 8.5
uv run python summarize_run.py --help
```

No more editing the script to change a file name. You get `--help`, type conversion and a clear error for free.



The debugger finds the bug without ten print lines

1

SET A BREAKPOINT

Click left of a line number, a red dot appears. The script will pause **before** that line.

2

START DEBUGGING

Press `F5`, or the arrow next to the play button, **Python Debugger: Debug Python File**

3

LOOK

Hover over a name, or read the **Variables** panel. Type any expression in the **Debug Console**.

4

STEP

Next line: `F10`. Into a function: `F11`. Out of it: `Shift+F11`. Continue: `F5`.

> Right-click a breakpoint to add a condition, such as `pressure_bar > 9`: stop only at the row that matters.



datetime gets durations and date formats right

```python
from datetime import datetime, timedelta

start = datetime.fromisoformat("2025-03-05 14:30:00")       # text to datetime
end = start + timedelta(minutes=45)                         # 2025-03-05 15:15:00

print((end - start).total_seconds())                        # 2700.0
print(start < end)                                          # True
print(start.strftime("%d/%m/%Y %H:%M"))                     # 05/03/2025 14:30
print(datetime.strptime("05/03/2025 14:30", "%d/%m/%Y %H:%M"))   # and back
```

- **datetime**: a moment. **timedelta**: a duration, the difference between two moments
- **strftime** writes text, **strptime** reads it, with a format you choose

> Store and exchange timestamps as ISO 8601 text (`2025-03-05T14:30:00`). `05/03/2025` is March in Belgium and May in the US.



Decode the short syntax in generated code

You read	It means
`sorted(runs, key=lambda run: run["pressure_bar"])`	**lambda**: a small function without a name, here "sort by pressure"
`for p, t in zip(pressures_bar, temperatures_c):`	**zip**: walk through two lists side by side
`status = "high" if pressure_bar > 9.0 else "ok"`	An **if** inside one expression
`def plot(*args, **kwargs):`	Accept any positional and any named arguments
`summarize(**settings)`	Unpack a dict into named arguments
`limit_bar: float | None = None`	A float or nothing, and nothing is the default
`if (n := len(rows)) > 100:`	**Walrus**: assign and use the value in one go
`match command:` with `case "start":`	Pick a branch by the shape of a value, like `switch`

> You do not have to write these. If a line is too dense to review, ask for the plain version.

